UNITÀ 4 · APPLICAZIONI PER SISTEMI MOBILI · LEZIONE 2

Il sistema operativo Android

Un kernel Linux, un runtime che non è una JVM, app chiuse in gabbia e un ciclo di vita deciso dal sistema. Ecco su che cosa girerà la nostra app.

5G TPSI Architettura, ciclo di vita, permessi, distribuzione

PER COMINCIARE

Che sistema operativo ha il tuo telefono?

SONDAGGIO

- **A** Android
- **B** iOS
- **C** Un'altra cosa
- **D** Non lo so con certezza

LE RADICI

Tutto parte da un hobby del 1991

> **Sto facendo un sistema operativo (libero), solo un hobby, non sarà grande e professionale come GNU.**

Linus Torvalds, studente all'Università di Helsinki, messaggio al gruppo comp.os.minix, 25 agosto 1991

2003

Nasce Android Inc., in California.

2005

Google compra la società.

2007

Annuncio con la Open Handset Alliance.

2008

Primo telefono: HTC Dream.

L'ARCHITETTURA

Cinque livelli, dal silicio alle app

App	App di sistema e app installate: Telefono, Fotocamera, la nostra QuizTPSI
Framework Java API	Activity Manager, Package Manager, View System, notifiche, Content Provider
ART + librerie native	Android Runtime che esegue il bytecode DEX; librerie C/C++ come SQLite, OpenGL ES, WebKit/Chromium
HAL	Hardware Abstraction Layer: interfacce standard per fotocamera, audio, sensori, Bluetooth
Kernel Linux	Driver, processi, memoria, gestione energetica, comunicazione Binder tra processi

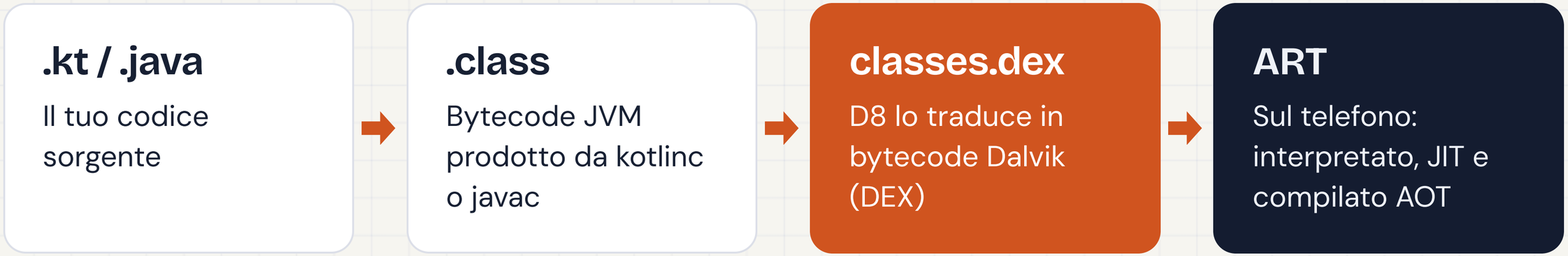
PUZZLE

Tocchi "Scatta": metti in ordine il viaggio della richiesta

1. L'HAL comunica con il driver nel kernel Linux
2. Il driver comanda il sensore e i dati dell'immagine tornano su
3. L'app chiama le API Camera del framework
4. Il framework inoltra la richiesta al servizio di sistema della fotocamera via Binder
5. Il servizio usa l'HAL della fotocamera del produttore

IL RUNTIME

ART non è una Java Virtual Machine



- ART ha sostituito la vecchia macchina virtuale Dalvik con Android 5.0, nel 2014.
- Da Android 7 combina JIT e compilazione anticipata dei metodi più usati, guidata da profili.

METTITI ALLA PROVA

Che cosa esegue davvero ART sul telefono?

QUIZ

A I file .class, come la JVM del PC

B Il sorgente Kotlin, interpretato riga per riga

C Bytecode DEX, compilato in parte in codice macchina

D Codice JavaScript prodotto da Kotlin

SICUREZZA DI BASE

Ogni app nella sua gabbia

- All'installazione ogni app riceve un proprio **utente Linux** (UID).
- Gira in un processo separato: i suoi file in `/data/data/<pacchetto>` sono privati.
- **SELinux** limita anche ciò che i processi di sistema possono fare.
- Ogni app è **firmata** dallo sviluppatore: gli aggiornamenti devono avere la stessa firma.
- Per uscire dalla gabbia servono **permessi** e API del sistema.

Collegamento

È il principio del **privilegio minimo** visto in principi-sicurezza, applicato a milioni di telefoni.

Il "root" del telefono rompe questa gabbia: per questo molte app bancarie si rifiutano di funzionare.

PUZZLE · COM'È FATTA UN'APP

Abbina i componenti di un'app Android

Activity	---	Reagisce a eventi di sistema, come "batteria scarica"
Service	---	Una schermata con cui l'utente interagisce
BroadcastReceiver	---	Lavoro senza interfaccia, per esempio la musica in riproduzione
ContentProvider	---	Il messaggio che chiede di avviare un componente
Intent	---	Condivide dati con altre app, come fa la rubrica
AndroidManifest.xml	---	Dichiara componenti e permessi dell'app

IL CICLO DI VITA

Non sei tu a decidere quando l'Activity vive

Che cosa fa l'utente	Chiamate che riceve l'Activity
Apre l'app	`onCreate → onStart → onResume`
Preme Home	`onPause → onStop`
Torna nell'app	`onRestart → onStart → onResume`
Ruota il telefono	`onPause → onStop → onDestroy → onCreate → onStart → onResume`
Preme Indietro	`onPause → onStop → onDestroy`

> Con poca memoria il sistema può **uccidere il processo** di un'app in background senza altre chiamate: ciò che conta va salvato in `onPause` o `onStop`.

QUIZ A TRABOCCHETTO

Un contatore è in una variabile dell'Activity e vale 7. Ruoti il telefono: che cosa succede?

QUIZ

- **A** Niente: cambia solo la disposizione dello schermo
- **B** L'Activity viene ricreata e il contatore torna al valore iniziale
- **C** L'Activity va in pausa e riprende con il contatore a 7
- **D** L'app si chiude con un errore

AL COMPUTER · 20 minuti

Simula il ciclo di vita in Java

1. Crea **CicloVita.java** con un `enum Stato { CREATA, AVVIATA, ATTIVA, FERMA, DISTRUTTA }`.
2. Scrivi i metodi `apri()`, `home()`, `torna()`, `ruota()`, `indietro()`: ognuno stampa i callback della tabella e aggiorna lo stato.
3. Conta i callback stampati con un campo `int chiamate`.
4. Nel `main` esegui: apri, home, torna, ruota, indietro.

Controllo: in tutto **17** callback; l'ultimo è `onDestroy` e lo stato finale è DISTRUTTA. Se ruota() ne stampa meno di 6, rivedi la tabella.

I PERMESSI

Dichiarati nel manifest, alcuni chiesti al momento

```
<!-- AndroidManifest.xml -->
<uses-permission android:name=
    "android.permission.INTERNET"/>
<uses-permission android:name=
    "android.permission.CAMERA"/>
```

- **Normali** (INTERNET, VIBRATE): concessi all'installazione.
- **Pericolosi** (CAMERA, posizione, microfono, contatti): chiesti **a runtime**, da Android 6.0.

```
// In Kotlin, nell'Activity
val richiesta = registerForActivityResult(
    ActivityResultContracts.RequestPermission()
) { concesso ->
    if (concesso) apriFotocamera()
    else spiegaPerche()
}
// quando l'utente tocca "Scatta"
richiesta.launch(Manifest.permission.CAMERA)
```

METTITI ALLA PROVA

Quale permesso l'app riceve all'installazione, senza chiedere nulla?

QUIZ

- **A** CAMERA
- **B** ACCESS_FINE_LOCATION
- **C** POST_NOTIFICATIONS, su Android 13 e successivi
- **D** INTERNET

AL TELEFONO O ALL'EMULATORE · 15 minuti

Indagine sui permessi del tuo telefono

1. In Impostazioni apri **Info sul telefono**: annota versione di Android e data della patch di sicurezza.
2. Cerca "autorizzazioni" nelle Impostazioni e apri la **Gestione autorizzazioni**.
3. Apri **Posizione**: conta le app "sempre consentite" e "solo durante l'utilizzo".
4. Apri il **Pannello privacy**, se c'è: quali app hanno usato microfono e fotocamera nelle ultime 24 ore?
5. Scrivi tutto in **permessi.txt** sul PC.

Controllo: il Pannello privacy esiste da Android 12. Se trovi un'app con la posizione "sempre consentita" senza motivo, hai trovato la prima cosa da sistemare.

LA DISTRIBUZIONE

Dal codice al Play Store



- **APK**: il pacchetto installabile, un archivio zip firmato.
- **AAB**: obbligatorio per le nuove app su Google Play da agosto 2021.
- Play chiede di puntare a una versione recente di Android (targetSdk).
- Fuori dallo store: altri store, come F-Droid, o installazione manuale.

AL COMPUTER · 15 minuti

Apri un APK come un archivio

1. Su **f-droid.org** scegli un'app open source e scarica il suo file .apk sul PC, senza installarlo.
2. Aprilo con 7-Zip, oppure copialo e rinomina la copia in **.zip**.
3. Trova `classes.dex`, `AndroidManifest.xml`, `resources.arsc`, `res/` e, se c'è, `lib/`.
4. Apri il manifest con il Blocco note: che cosa vedi?
5. Annota cartelle e dimensioni in **dentro-apk.txt**.

Controllo: c'è almeno un file `classes.dex`, non ci sono file .class. Il manifest è illeggibile: nell'APK è salvato in XML binario.

IL SISTEMA OGGI

AI e sicurezza dentro il sistema operativo





BIGLIETTO D'USCITA

Completa prima di uscire

Il livello più basso di Android è il kernel _____.

Il runtime che esegue il bytecode DEX si chiama _____.

Quando ruoti il telefono l'Activity riceve per ultima la chiamata _____.

I permessi pericolosi si chiedono a _____, da Android 6.0.

Su Google Play le nuove app si caricano in formato _____.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- Kernel Linux, HAL, ART e librerie, framework, app
- Ogni app ha il suo utente Linux: sandbox
- Activity, Service, Receiver, Provider, Intent
- Ciclo di vita: la rotazione ricrea l'Activity
- Permessi normali e pericolosi, chiesti a runtime
- APK installabile, AAB per Google Play

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.